



Summer School on RISC-V Hardware–Software Co-Design

RISC-V Vector Toolchains and Firmware

Mushafia Sadia Aman | Research Associate
Centre for AI and Big Data (CAID), Namal University Mianwali

Contents

1 Cross-Compilation Toolchains

2 RISC-V Vector (RVV) Compilation

3 Bare-Metal Firmware and Boot Flow

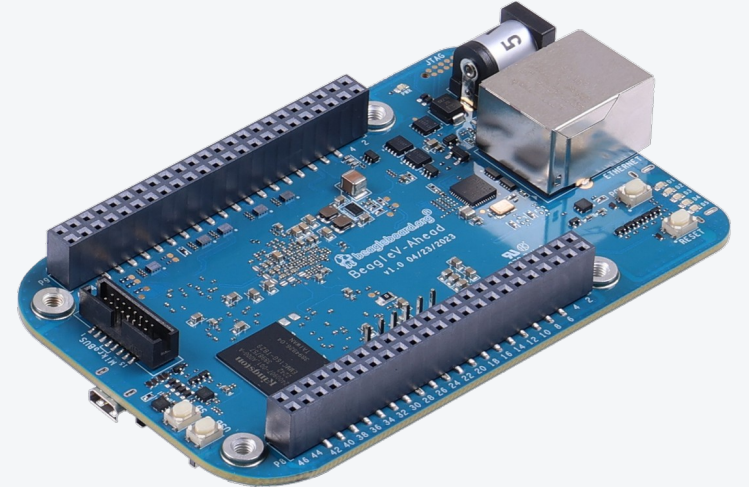
4 Accelerator Interfaces and Drivers

5 Accelerator-Aware Optimization

6 Debugging, Profiling, and Validation

Essential Terms Before We Start

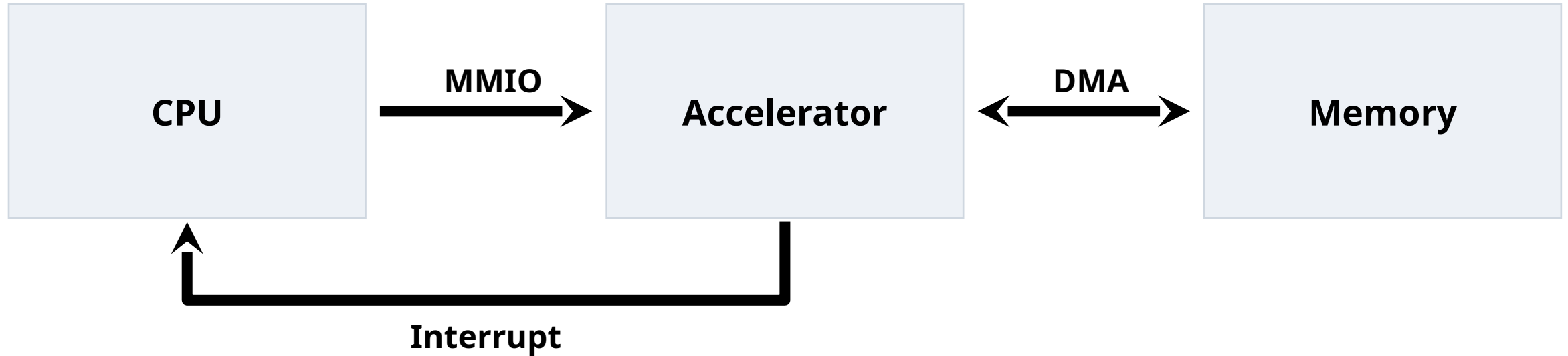
- **ISA:** Software-visible instructions; RISC-V is the ISA and RVV extends it.
- **Microarchitecture:** The hardware implementation of an ISA; the same ISA can perform very differently.
- **Toolchain:** Compiler, assembler, linker and binary utilities that build software for the target.
- **Firmware:** Low-level code that initializes and directly controls the hardware.
- **Accelerator:** Specialized hardware that takes over a computational task from the CPU.
- **Driver:** The software interface that configures the accelerator and exchanges data.



A REAL RISC-V PLATFORM

ISA	RISC-V
MICROARCHITECTURE	TH1520 / C010
ACCELERATOR	4-TOPS NPU

Accelerator Communication Basics



MECHANISM	DIRECTION	WHAT THE DRIVER DOES
MMIO	CPU to accelerator	Write CTRL to start a job, read STATUS to follow progress
DMA	Memory and accelerator	Set source, destination and length; the transfer runs without CPU copies
Interrupt	Accelerator to CPU	Signals completion, so the CPU does not poll STATUS while the job runs



Cross-Compilation Toolchains

Build, configure, and verify software for the RISC-V target.



Cross-Compilation: Host and Target

Cross-compilation builds software on a host system for execution on a different target system.



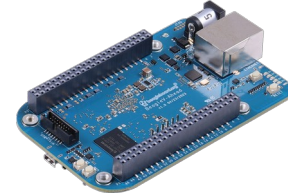
HOST SYSTEM

Workstation or server



CROSS-COMPILER

RISC-V compiler



TARGET SYSTEM

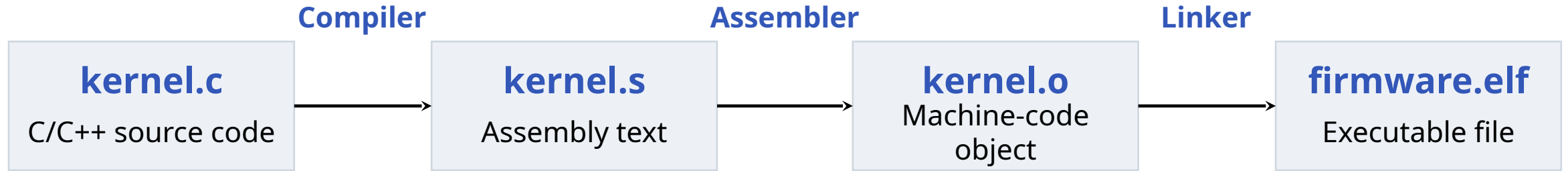
**RISC-V development
board**

Toolchain prefix: target identifier placed before each tool name.

TOOLCHAIN PREFIX	FULL COMPILER NAME	TARGET ENVIRONMENT
<code>riscv64-unknown-elf-</code>	<code>riscv64-unknown-elf-gcc</code>	Bare-metal RISC-V (no operating system)
<code>riscv64-linux-gnu-</code>	<code>riscv64-linux-gnu-gcc</code>	RISC-V Linux user space

Compilation Pipeline

STAGES AND GENERATED FILES



STAGE	OPERATION	OUTPUT
Compiler	Translates C/C++ into assembly	Assembly file (.s)
Assembler	Encodes assembly instructions as machine code	Object file (.o)
Linker	Combines object files and required libraries	Executable and Linkable Format (ELF) file

REPRESENTATIVE TOOL COMMANDS

```
$ riscv64-unknown-elf-gcc -S kernel.c -o kernel.s      # compile
$ riscv64-unknown-elf-as kernel.s -o kernel.o          # assemble
$ riscv64-unknown-elf-ld kernel.o -o firmware.elf      # link
```

RISC-V ISA Extensions

COMPILER OPTION

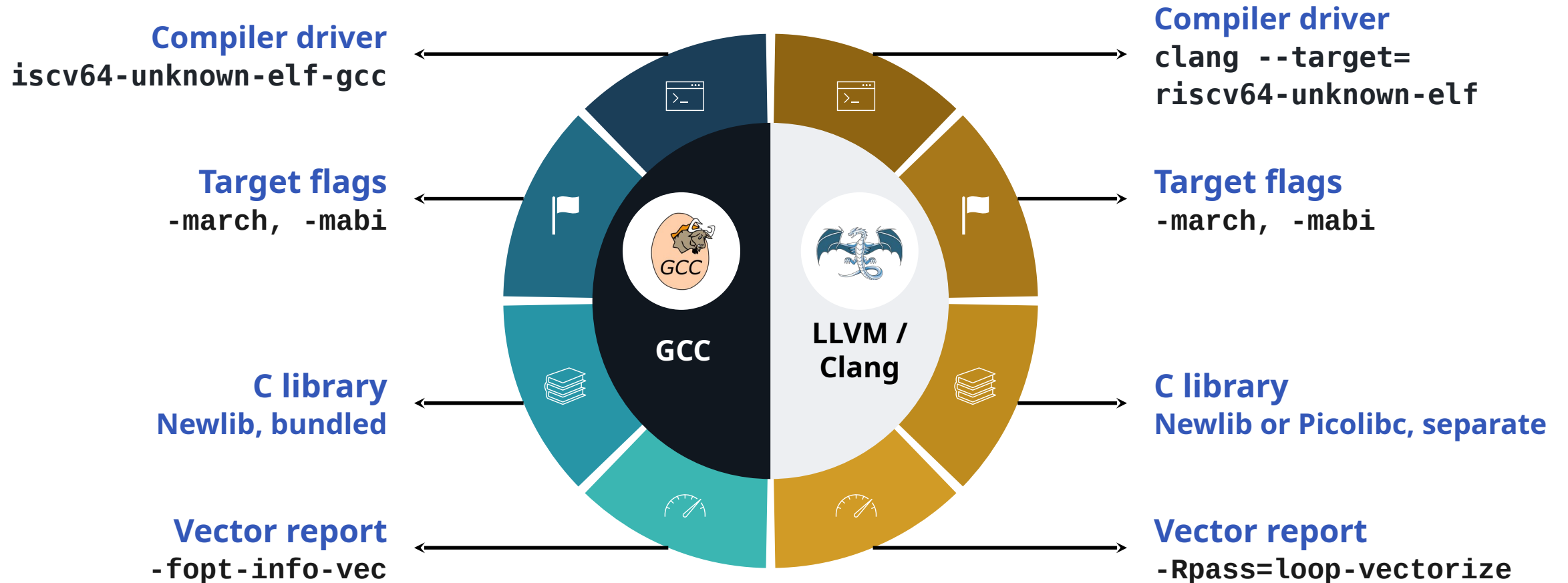
-march=	rv64	g	c	v
----------------	-------------	----------	----------	----------

TOKEN DEFINITIONS

TOKEN	BASIC DEFINITION
rv64	64-bit RISC-V architecture with the base integer instruction set.
g	General-purpose abbreviation: I (integer), M (multiply/divide), A (atomic memory operations), F/D (floating point), Zicsr (control and status register access), and Zifencei (instruction-fetch synchronization).
c	Compressed 16-bit instruction encodings that reduce program size.
v	Vector instructions that operate on multiple data elements per instruction.

Compiler constraint: -march=rv64gcv permits these instructions; the target processor must implement every selected extension.

GCC and LLVM Toolchains



GNU Toolchain Installation and Verification

Build the bare-metal toolchain

```
$ git clone https://github.com/riscv-collab/riscv-gnu-toolchain
$ cd riscv-gnu-toolchain
$ ./configure --prefix=/opt/riscv \
  --with-arch=rv64gcv --with-abi=lp64d
$ make -j$(nproc)
$ export PATH=/opt/riscv/bin:$PATH
$ export CROSS=riscv64-unknown-elf-
```

Verify the installed toolchain

1 Compiler target

```
${CROSS}gcc -dumpmachine
```

```
riscv64-unknown-elf
```



2 ELF ISA attributes

```
${CROSS}readelf -A firmware.elf
```

```
Tag_RISCV_arch:
rv64..._v1p0...
zvl256b1p0
```



3 RVV disassembly

```
${CROSS}objdump -d firmware.elf
```

```
vsetvli a5,a0,e32,m1
vle32.v v8,(a1)
```

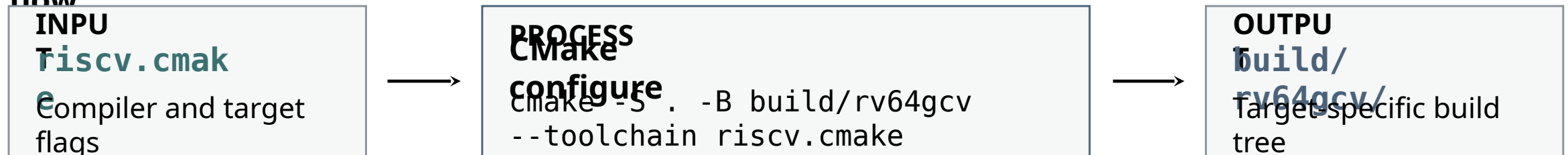
CMake Toolchain File

Toolchain file

CMake configuration loaded before project setup to select target compilers and flags.

```
# riscv.cmake
set(CMAKE_SYSTEM_NAME Generic)           # Generic = bare-metal target
set(CMAKE_SYSTEM_PROCESSOR riscv64)
set(CMAKE_TRY_COMPILE_TARGET_TYPE STATIC_LIBRARY) # static-library compile checks
set(CROSS riscv64-unknown-elf)
set(CMAKE_C_COMPILER ${CROSS}-gcc)
set(CMAKE_ASM_COMPILER ${CROSS}-gcc)
set(CMAKE_OBJCOPY ${CROSS}-objcopy)
set(ARCH_FLAGS "-march=rv64gcv -mabi=lp64d -mmodel=medany")
set(CMAKE_C_FLAGS_INIT "${ARCH_FLAGS}")
set(CMAKE_ASM_FLAGS_INIT "${ARCH_FLAGS}")
# link.ld defines the target memory layout
set(CMAKE_EXE_LINKER_FLAGS_INIT
    "-T${CMAKE_CURRENT_LIST_DIR}/link.ld -nostartfiles")
```

Configuration flow



Demo1: Cross-Compile and Verify the Target

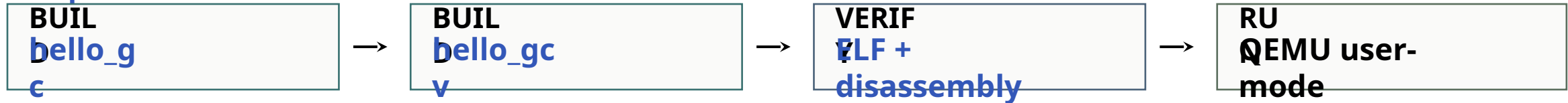
Goal Build one C program for RV64GC and RV64GCV, then verify the ISA and ABI recorded in each ELF.

Linux-target compiler

Produces Linux RISC-V executables.

QEMU user-mode

Runs them without a full virtual board.



Commands

```
riscv64-linux-gnu-gcc -O2 -static \  
-march=rv64gc -mabi=lp64d hello.c -o  
hello_gc  
riscv64-linux-gnu-gcc -O2 -static \  
-march=rv64gcv -mabi=lp64d hello.c -o  
hello_gcv  
  
riscv64-linux-gnu-readelf -hA hello_gc  
hello_gcv  
riscv64-linux-gnu-objdump -d hello_gcv | head  
-80  
  
qemu-riscv64 ./hello_gc  
qemu-riscv64 ./hello_gcv
```

Evidence

1. **ELF metadata:** ISA and ABI fields
2. **Disassembly:** rv64gcv instruction mix
3. **Runtime:** identical program output

```
smarz@DOM0:~/Develop/blog/risc_v $ readelf -h src/userspace/helloworld.elf  
ELF Header:  
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00  
  Class:                                ELF64  
  Data:                                      2's complement, little endian  
  Version:                               1 (current)  
  OS/ABI:                                UNIX - System V  
  ABI Version:                           0  
  Type:                                  EXEC (Executable file)  
  Machine:                               RISC-V  
  Version:                               0x1  
  Entry point address:                   0x101e4  
  Start of program headers:              64 (bytes into file)  
  Start of section headers:             14072 (bytes into file)  
  Flags:                                  0x4, double-float ABI  
  Size of this header:                   64 (bytes)  
  Size of program headers:               56 (bytes)  
  Number of program headers:              2  
  Size of section headers:               64 (bytes)  
  Number of section headers:             11  
  Section header string table index:     10  
smarz@DOM0:~/Develop/blog/risc_v $
```



RISC-V Vector (RVV) Compilation

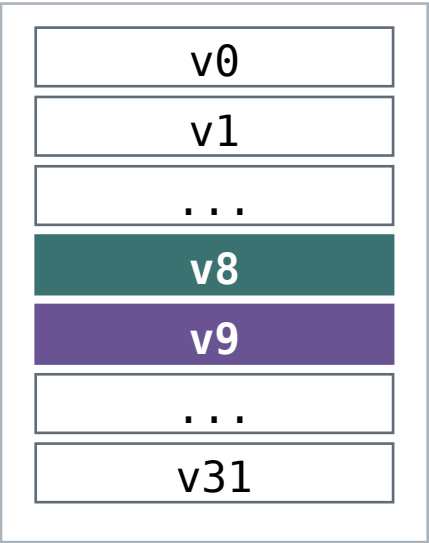
Understand the RVV model and generate efficient vector code.



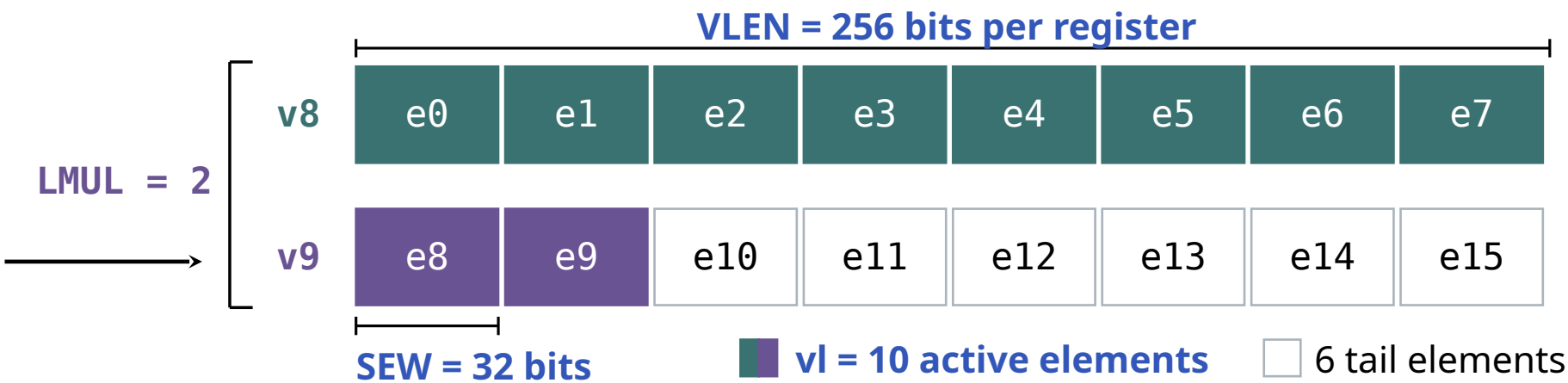
RVV Programming Model

Vector register file 32 registers (v0-v31); VLEN is the hardware-defined width of each register.

Register file

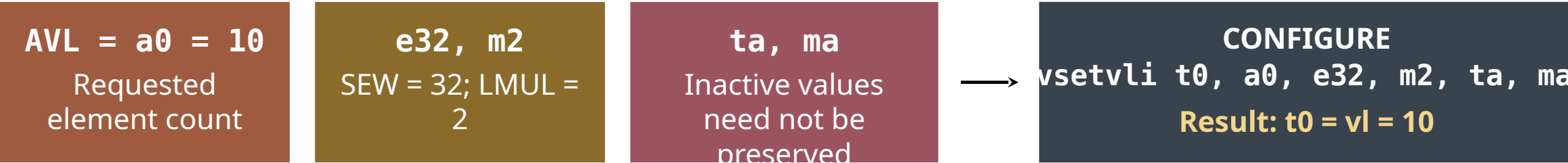


Example: register group and active elements



v8 + v9 form LMUL = 2

Vector configuration



Vector-Length Agnostic Loops

Vector-length agnostic (VLA) one binary operates correctly for different hardware VLEN values.

Strip-mined loop: each iteration requests the remaining count; vsetvl returns the vl to process.

Example: N = 20; VLEN = 128, SEW = 32, LMUL = 2; VLMAX = 8

Iteration 1

AVL = 20 -> vl = 8



20 - 8 = 12 remain

Iteration 2

AVL = 12 -> vl = 8



12 - 8 = 4 remain

Iteration 3

AVL = 4 -> vl = 4



4 - 4 = 0 remain

RVV intrinsic loop

The final iteration uses a smaller vl; no scalar remainder loop is required.

```
1 void saxpy(size_t n, float a, float *x, float *y) {
2     while (n > 0) {
3         size_t vl = __riscv_vsetvl_e32m2(n);
4         vfloat32m2_t vx = __riscv_vle32_v_f32m2(x, vl);
5         vfloat32m2_t vy = __riscv_vle32_v_f32m2(y, vl);
6         vy = __riscv_vfmacc_vf_f32m2(vy, a, vx, vl);
7         __riscv_vse32_v_f32m2(y, vy, vl);
8         n -= vl; x += vl; y += vl;
9     }
10 }
```

Demo2: Run One RVV Binary at Three Vector Lengths

Objective

compile one SAXPY binary and run it correctly at VLEN = 128, 256 and 512.

Mechanism

vsetvl reports the vl the hardware can process; the loop consumes vl elements per iteration.

1 Build and run

```
1 $ riscv64-linux-gnu-g++ -O3 -static -march=rv64gcv -mabi=lp64d \  
2     saxpy_rvv.cpp -o saxpy_rvv \  
3 $ for V in 128 256 512; do \  
4     qemu-riscv64 -cpu rv64,v=true,vlen=$V,vext_spec=v1.0 ./saxpy_rvv; done \  
5 # print vl each iteration to observe the tail
```

2 Measured vl sequence: N = 20, LMUL = 2, SEW = 32

VLEN	VLMAX	vl sequence per iteration	Iterations	Numerical output
128	8	8, 8, 4	3	identical
256	16	16, 4	2	identical
512	32	20	1	identical

Masks and Predication

Mask register

a vector register whose bits select which lanes an instruction may write.

Predication

evaluating both sides of a condition and committing only the lanes the mask allows.

1 Lane view: v0 decides which lanes commit

v8	input lanes	3.1	9.8	0.4	7.2	5.0	1.6	8.4	2.9
v0	mask bits	1	0	1	0	0	1	0	1
result		updated	kept	updated	kept	kept	updated	kept	updated

2 Masked arithmetic: no branch in the loop body

```
1 vmflt.vf v0, v8, fa0    # v0 = x[i] < limit
2 vfadd.vf v12, v8, fa1, v0.t
3 vcpop.m  t0, v0         # matched lanes
```

3 Policies and typical use

ta / tu	tail policy: elements past vl are agnostic or left undisturbed.
ma / mu	mask policy: the same choice for lanes the mask turned off.
Best used for	if/else in loops, clamping, image edge rows, sparse updates.

Vector Memory Access Patterns

Access pattern

the address sequence a vector load generates; the instruction must match the layout.

Data layout

how the array is arranged in memory; normally chosen by the programmer, not by the kernel.

1 Load and reduction instructions by access pattern

Instruction	Pattern	Address sequence	Cost and use
vle32.v	Unit stride	$a[i], a[i+1], a[i+2]$	Fastest path: design the data so this is the instruction you use.
vlse32.v	Constant stride	$a[i * 8]$	One column of a row-major matrix; costs far more than unit stride.
vluxei32.v	Indexed gather	$a[idx[i]]$	Arbitrary offsets: the most flexible and the slowest option.
vlseg3e32.v	Segment load	x, y, z interleaved	De-interleaves an array of structs into separate registers at no extra cost.
vfredusum.vs	Reduction	sum of a vector	Vector to scalar; changes the order of floating-point addition.

Demo3: Benchmark Unit-Stride, Strided and Gather Loads

Objective

measure how data layout changes vector efficiency using three kernels with the same result.

Measurement

cycles per element from gem5 or hardware counters; QEMU wall time is not a performance figure.

1 Build and run

```
1 # kernels: unit a[i], stride a[i*8], gather a[idx[i]]
2 $ make rvv
3 $ for M in unit stride gather; do \
4     ./bench --mode $M --n 1048576; done
5 # report cycles/element per mode, not elapsed wall time
```

2 Cycles per element: N = 1,048,576

Access pattern	Load instruction	Cycles per element	Output checksum
unit	vle32.v	baseline	identical
stride 8	vlse32.v	measured	identical
gather	vluxe32.v	measured	identical

Three Routes to Vector Code

Scope

three ways to get vector instructions into a binary, in increasing order of effort.



You write	plain C; the compiler finds the loops	<code>__riscv_v*</code> calls from C	the instruction sequence itself
Trade-off	Portable across ISAs and VLENs; gives up silently on messy loops.	You control LMUL, masks and layout; ties the source to RVV.	Full control and custom instructions; costly to test and maintain.
Use when	first attempt, always	the measured hot kernel	last resort or a new extension

Did the Compiler Actually Vectorise?

Scope

four checks that confirm vector instructions were generated and actually executed.

1 The four checks, in order

	Check	Command	What confirms it
01	Read the disassembly	<code>objdump -d saxpy.o \</code> <code> grep -E "vset v[ls]e vf"</code>	<code>vsetvli</code> , <code>vle32.v</code> and <code>vfmacc.vf</code> are present
02	Run it in a simulator	<code>qemu-riscv64 -cpu \</code> <code>rv64,v=true,vlen=256 ./saxpy</code>	QEMU for speed, Spike for reference semantics
03	Count what executed	<code>spike -g --isa=rv64gcv \</code> <code>pk ./saxpy 2> hist.txt</code>	an instruction histogram, not wall clock time
04	Compare against scalar	<code>perf stat -e \</code> <code>cycles,instructions ./saxpy</code>	a speed-up in cycles per element on the same input



Bare-Metal Firmware and Boot Flow

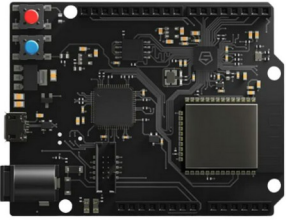
Bring the processor from reset to a reliable runtime.



Bare Metal or Linux?

Bare metal

Minimal stack - direct hardware control



HiFive1 Rev B

FE310 RISC-V microcontroller
Typical bare-metal platform

Two layers

your code + board support

Deterministic

no scheduler or MMU; repeatable cycle counts

Bring-up + benchmarks

clean, repeatable measurements

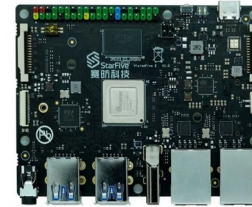
Software stack

Timing

Best use

Linux or an RTOS

Managed stack - deployment services



VisionFive 2

64-bit RISC-V single-board computer
Ubuntu-supported platform

Five layers

application + libc + kernel + driver + hardware

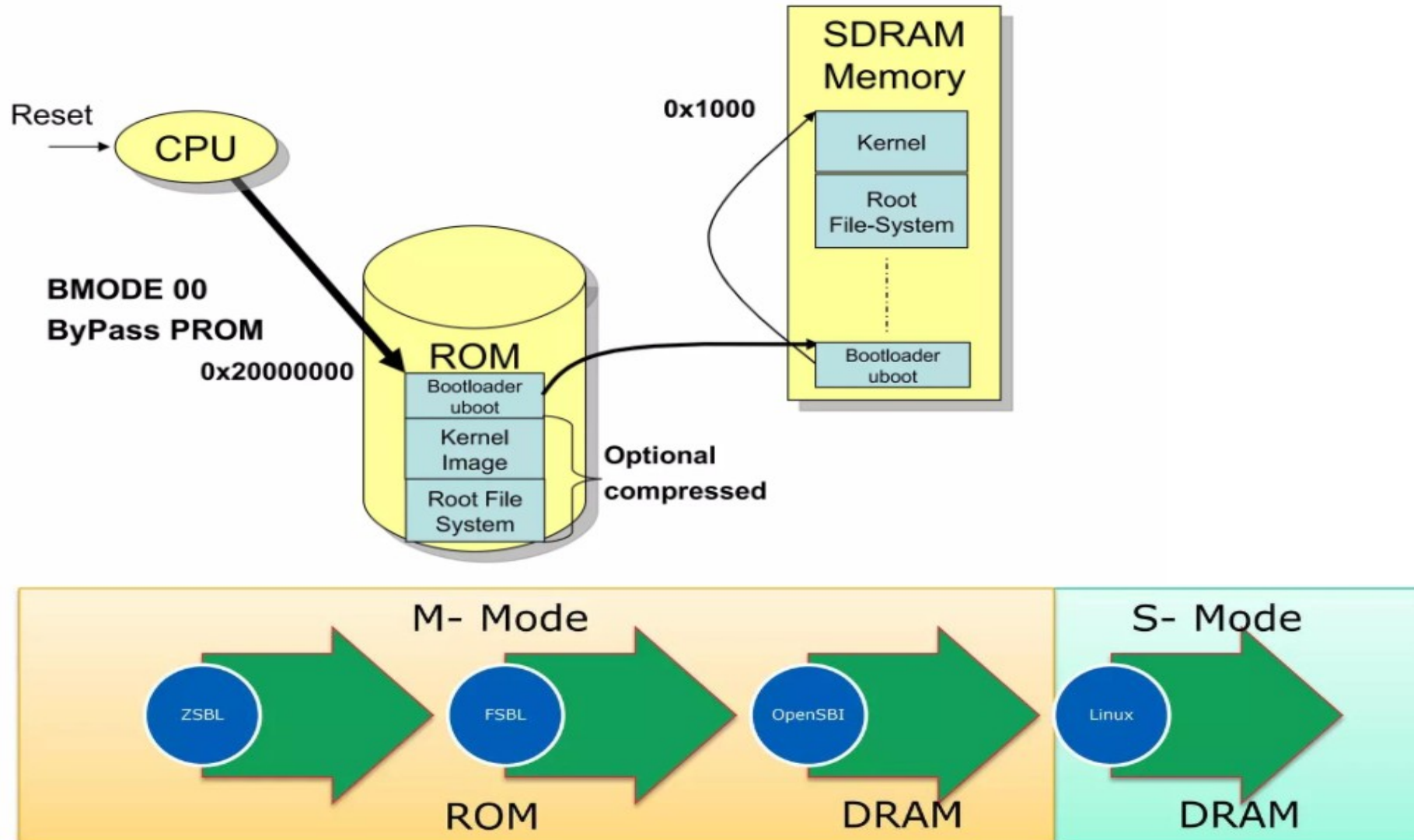
Jitter

scheduling + interrupts + paging

Frameworks + products

managed services and edge deployment

The RISC-V Boot Flow



Demo4: Boot Bare-Metal and Print via UART

Success condition

the UART prints before the workload begins.

```
$ qemu-system-riscv64 -nographic -machine virt -bios none -kernel hello
Hello World!
```

Minimum build and run path

COMMON FLAGS **-march=rv64gcv** **-mabi=lp64d**

```
gcc -c crt0.S $COMMON
gcc -c main.c uart.c $COMMON -ffreestanding
gcc -T link.ld -nostartfiles \
    crt0.o main.o uart.o -o firmware.elf
qemu-system-riscv64 -M virt -nographic \
    -bios none -kernel firmware.elf
```

Expected output

01 BOOT ORDER

UART banner appears before the workload.

02 RUNTIME DATA

.data is initialised; .bss begins at zero.

03 LINK PLACEMENT

Map addresses agree with link.ld.

04 RECORD

Keep the ELF, map file and console log.

Before main(): crt0 and the Linker Script

crt0.S the assembly stub that runs before main and prepares the C environment.

link.ld declares the memory map and the symbols crt0 needs.

1 What crt0.S must do

Set sp and gp

```
la sp, _stack_top
la gp, __global_pointer$
```

Initialise memory

```
copy _sdata to _sdata .. _edata
zero _sbss .. _ebss
```

Enable FP and vector

```
csrs mstatus, (1<<13)|(1<<9)
```

Install the trap vector

```
csrw mtvec, trap_handler
```

Call main()

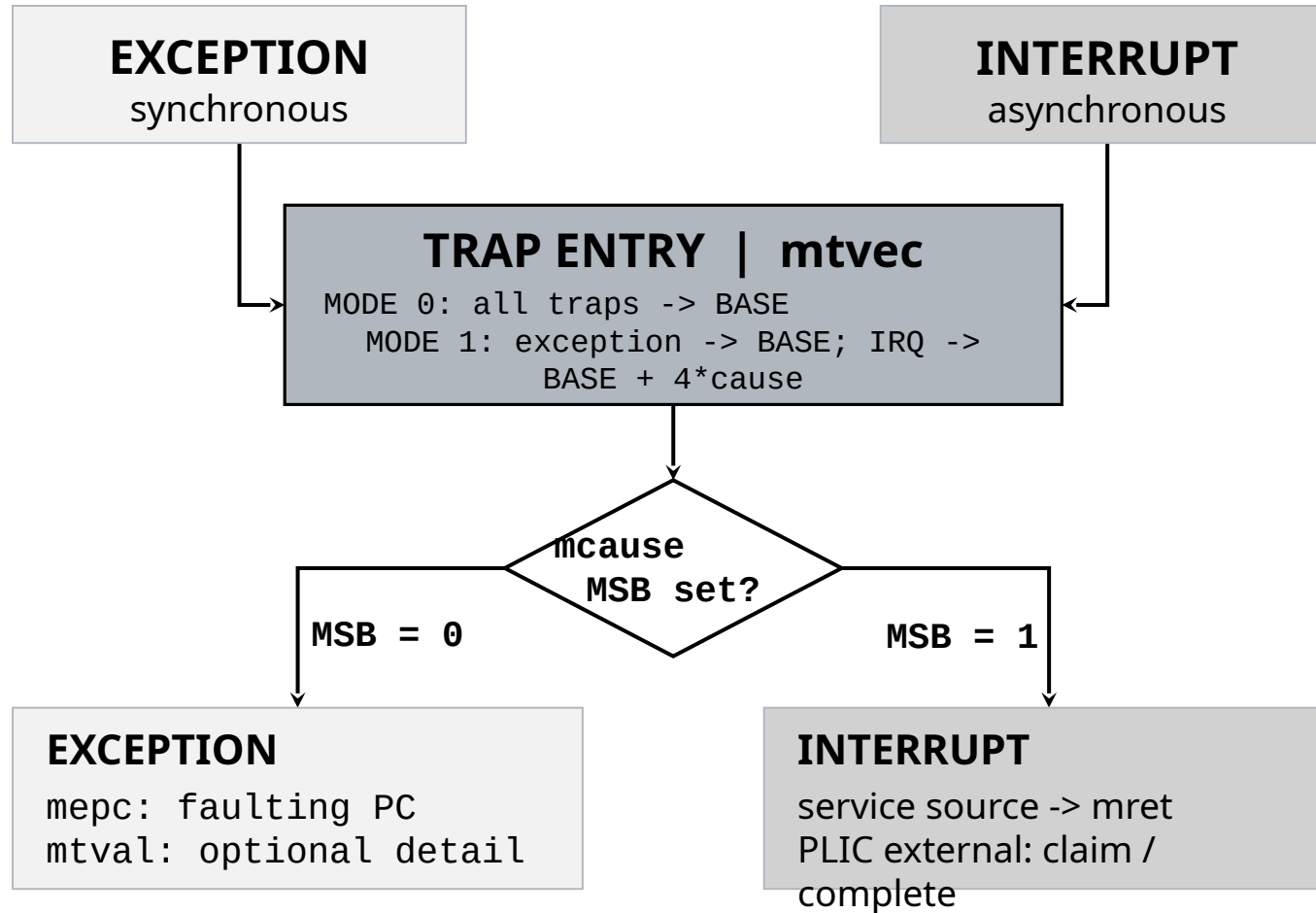
```
tail main
```

2 The memory map

```
1 MEMORY {
2   FLASH (rx) : ORIGIN = 0x20000000, LENGTH = 4M
3   RAM (rwx) : ORIGIN = 0x80000000, LENGTH =
4   128K
5 }
6 SECTIONS {
7   .text : { KEEP(*(.init)) *(.text*) } > FLASH
8   .data : { *(.data*) } > RAM AT> FLASH
9
10  .bss : { *(.bss*) } > RAM
11  _stack_top = ORIGIN(RAM) + LENGTH(RAM);
}
```

Traps and Interrupts

Trap entry and dispatch



The four CSRs

mtvec BASE + MODE

mcause interrupt bit + cause code

mepc saved PC

mtval optional exception detail

Demo5: Turn a Crash into Three Register Values

Objective trigger one illegal instruction, capture mcause, mepc and mtval, then find the fault.

Constraint insert the bad word once in the test path; remove it and confirm the firmware resumes.

1 Fault injection and inspection

```
1 // insert once in the test path
2 asm volatile (".word 0xffffffff");
3 $ riscv64-unknown-elf-objdump -d firmware.elf | less
4 $ riscv64-unknown-elf-gdb firmware.elf
5 (gdb) target remote :1234
6 (gdb) x/4i $mepc
```

2 Expected output

Check	Expected result
trap handler	prints the cause and the faulting PC
mepc	maps to the injected instruction in objdump or GDB
after removal	the firmware resumes its normal path

No Linux, no libc, no printf. Our own startup code initializes the machine, UART driver prints directly through MMIO.



Accelerator Interfaces and Drivers

Connect software to custom hardware through MMIO, DMA, and interrupts.



Describing the Accelerator

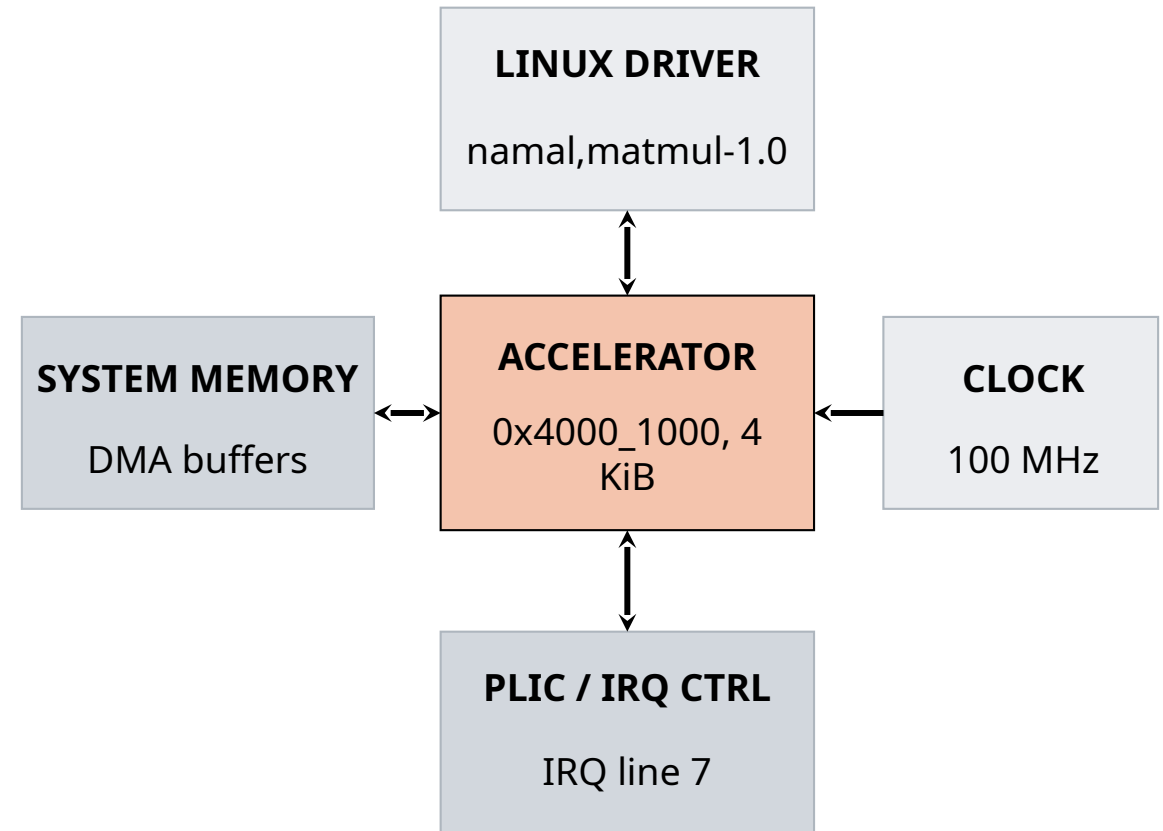
Device-tree node

```
matmul@40001000 {  
    compatible = "namal,matmul-1.0";  
    reg = <0x0 0x40001000 0x0 0x1000>;  
    interrupts = <7>;  
    dma-coherent;  
    clock-frequency = <100000000>;  
};
```

Property to resource mapping

- ❑ **compatible:** driver bind (namal,matmul-1.0)
- ❑ **reg:** MMIO window at 0x4000_1000, 4 KiB
- ❑ **interrupts:** PLIC line 7
- ❑ **dma-coherent:** cache-coherent DMA buffers
- ❑ **clock-frequency:** 100 MHz input clock

Hardware resources



The Register Map Is the Interface

Register map the accelerator's software interface: memory addresses with side effects.

Offsets illustrative only: always work from the datasheet or the device tree node.

1 The register map

Offset	Name	Access	Purpose
0x00	CTRL	RW	start, reset, irq_en
0x04	STATUS	RO	busy, done, error
0x08	SRC_ADDR	RW	input buffer address
0x10	DST_ADDR	RW	output buffer address
0x18	LENGTH	RW	elements or bytes
0x1C	CONFIG	RW	data type, tile, mode
0x20	CYCLES	RO	cycles of the last run

- **volatile**: the compiler otherwise hoists your polling loop away
- **fence**: device accesses follow the I/O bits; issue fence w,o before START
- **physical addresses**: the device does not see your virtual memory

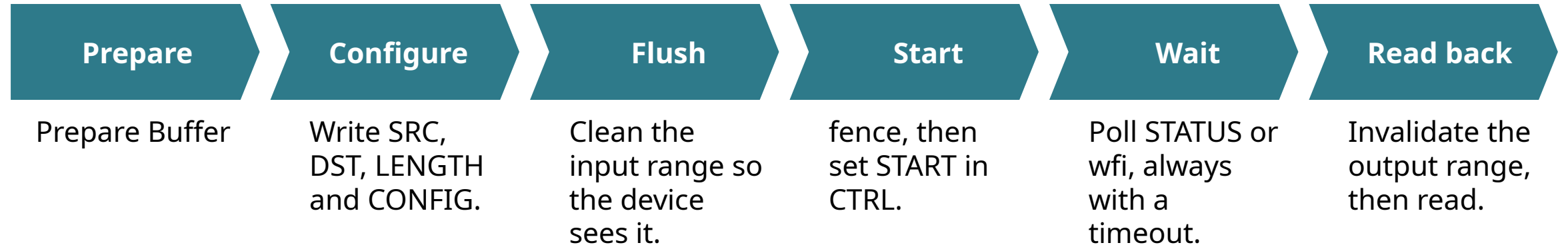
2 Reaching those registers safely

```
1 #define ACC 0x40001000UL
2 #define R(o) \
3     (*(volatile uint32_t*)(ACC+
4     (o)))
5 #define R64(o) \
6     (*(volatile uint64_t*)(ACC+
7     (o)))
8 R64(0x08) = src_phys; // 64-bit
9 R64(0x10) = dst_phys;
10 R(0x18) = n;
    asm volatile ("fence w,o");
    R(0x00) = CTRL_START; // start
```

The Life of One Accelerator Call

Accelerator call six steps from buffer to result; skipping the cache steps is the classic bug.

1 Six steps, in order



2 Three ways to wait

Method	Behaviour
Polling	lowest latency, burns a core; good for short operations and bring-up.
Interrupt	core sleeps on wfi; needs mstatus.MIE, mie.MEIE, a trap vector and a PLIC claim.
DMA and double buffering	transfer the next tile while the current one computes.

The Hazard Nobody Sees Coming

Cache coherence whether the device sees the CPU's writes and the CPU sees the device's writes.

Symptom the data is correct in memory, just not where the accelerator is looking.

1 How the bug happens

Step	What happens	Why the result breaks
1	CPU fills the buffer	the writes land in the data cache, not in DRAM
2	Driver starts the device	CTRL is written and the accelerator begins fetching
3	Device reads DRAM	it sees the old contents; the new bytes are still in cache
4	Result is wrong	the run looks like a hardware bug for the rest of the day

2 Three ways to make it correct

- **Clean before, invalidate after:** write back the input range, then invalidate the output range
- **Use uncached memory:** allocate shared buffers in a non-cacheable region; slower for the CPU
- **Let the hardware do it:** a coherent interconnect removes the problem; verify dma-coherent first



Accelerator-Aware Optimization

Balance computation, data movement, buffering, and offload overhead.



Six Techniques That Usually Pay

Scope six changes that reliably improve accelerator and vector throughput.

1 Techniques, in the order worth trying them

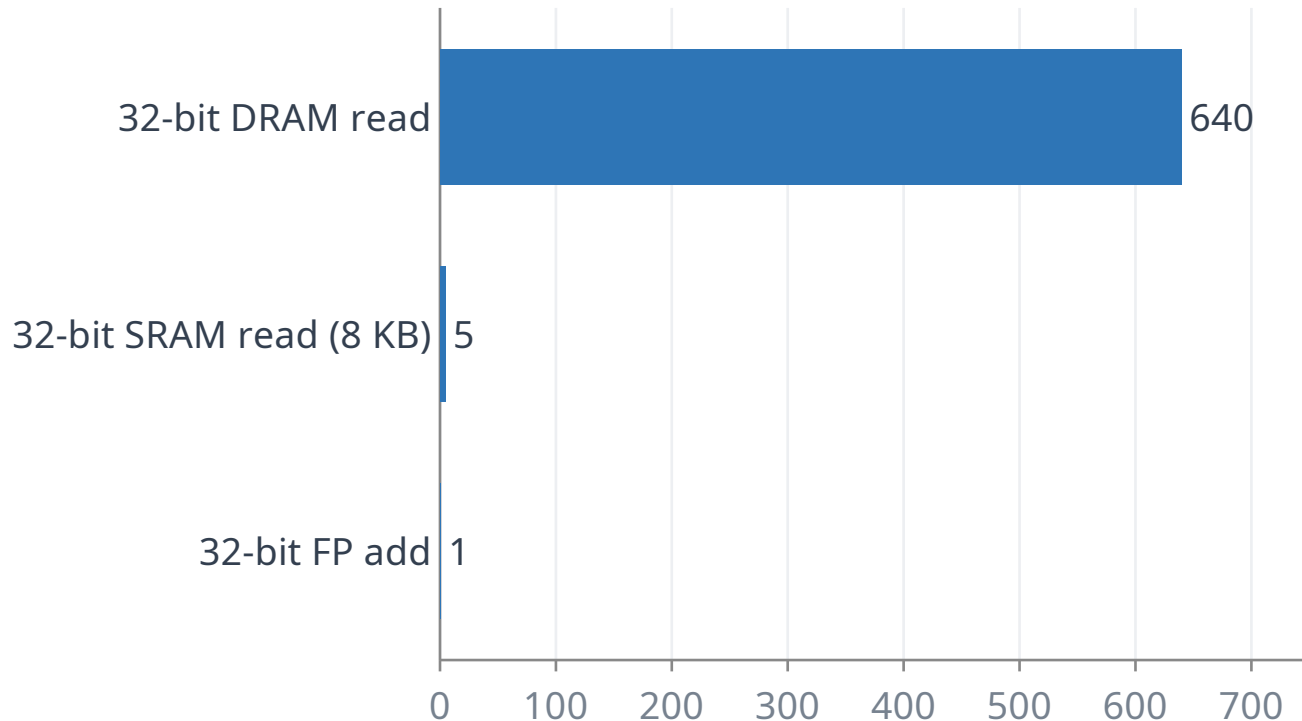
Technique	What to do
1 Keep the unit fed	double-buffer with DMA so the next tile loads while the current one computes
2 Amortise the setup	Combine many small operations into one larger batch.
3 Match the data layout	tile and pack data the way the datapath reads it; one conversion beats a stride penalty every step
4 Tune LMUL	larger LMUL means fewer loop iterations but fewer registers: measure m1, m2 and m4
5 Hoist vsetvli	if SEW and LMUL do not change, set them once outside the loop
6 Offload only what pays	break-even needs compute time above transfer plus setup time; Amdahl decides

Why Data Movement Dominates

Scope

relative energy per 32-bit operation on a 45 nm process; the ratio is the lesson.

1 Energy per 32-bit operation



2 What follows from this

Reuse beats recompute

a value already in registers costs almost nothing to use again.

Tile for the local memory

choose tiles that fit the scratchpad or L1, not the problem size.

Fuse operations

two passes over a large array cost two trips to DRAM.

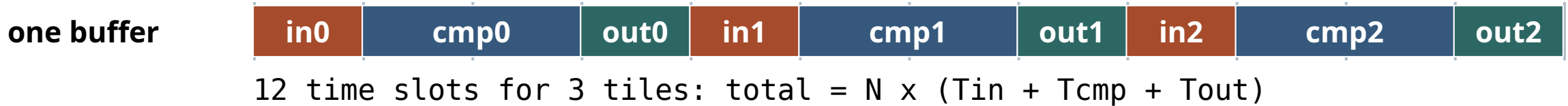
Stream, do not stall

overlap transfer with compute or the accelerator waits on memory.

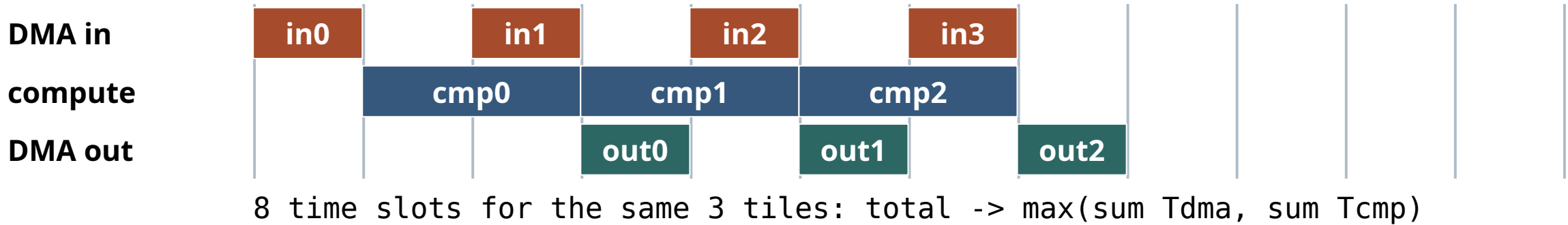
Double Buffering on a Timeline

Double buffering overlap the transfer of the next tile with the computation of the current one.

1 Single buffer: every stage waits for the last



2 Double buffer: the slower stream sets the pace



- **Two buffers, used alternately:** tile n computes from $\text{buf}[n \ \& \ 1]$ while tile $n+1$ loads into $\text{buf}[(n+1) \ \& \ 1]$
- **Issue the next transfer before you wait:** start the DMA for tile $n+1$, then poll for tile n
- **The ceiling is the slower stream:** if T_{dma} exceeds T_{cmp} the unit still starves, so widen the bus or shrink the tile

Is It Worth Offloading? The Break-Even Size

Break-even

the size n where offload time equals CPU time; below it the CPU wins.

1 Two cost models

Model	Time
CPU	n / R_{cpu}
Offload	$T_{setup} + 2n/B_{dma} + n/R_{acc}$

Crossover

$$n^* = T_{setup} / (1/R_{cpu} - 2/B_{dma} - 1/R_{acc})$$

2 Worked with measured numbers

- 1 $T_{setup} = 2000$ cycles
- 2 $R_{cpu} = 0.5$ elem/cycle
- 3 $B_{dma} = 4.0$ elem/cycle // each way
- 4 $R_{acc} = 16.0$ elem/cycle
- 5
- 6 $n^* = 2000 / (2.0 - 0.5 - 0.0625)$
- 7 $= 1391$ elements

Demo6: Break-Even Size and Double Buffering

Objective

measure scalar, RVV and accelerator paths across N and find where offload wins.

Constraint

same compiler, same data, same machine: only the mode changes.

1 Problem-size sweep

```
1 for N in 64 128 256 512 1024 2048 4096 16384 65536; do
2   for M in scalar rvv accel; do
3     ./bench --n $N --mode $M
4   done
5 done
6 # repeat with single and double buffering; record cycles/element
```

2 Expected output

Check	Expected result
crossover N*	the size where accelerator time drops below CPU time
LMUL or buffering	every change backed by measured data, not expectation
occupancy	separates a compute-bound run from a transfer-bound one



Profiling, and Validation

Isolate faults, measure performance, and prove numerical correctness.



Profiling on a RISC-V Target

1 Read the cycle counter

```
1 static inline uint64_t rdcycle(void) {
2     uint64_t c;
3     asm volatile ("csrr %0, cpucycle"
4                   : "=r"(c) ::
5                     "memory");
6     return c;    // "memory" stops motion
7 }
8 uint64_t t0 = rdcycle();
9 saxpy(n, a, x, y);
uint64_t cycles = rdcycle() - t0;
```

2 Counters worth reading

Counter	What it tells you
cpucycle	elapsed core cycles
minstret	instructions retired
mhpmcounter3..31	cache misses, stalls
CYCLES	the device's own busy time

3 Derive numbers you can defend

- **Cycles per element:** cycles / n, comparable across problem sizes and machines
- **IPC:** instret / cycle, whether the pipeline is actually using
- **Vector efficiency:** useful elements / (vl x iterations)
- **Accelerator occupancy:** device busy cycles / total wall cycles

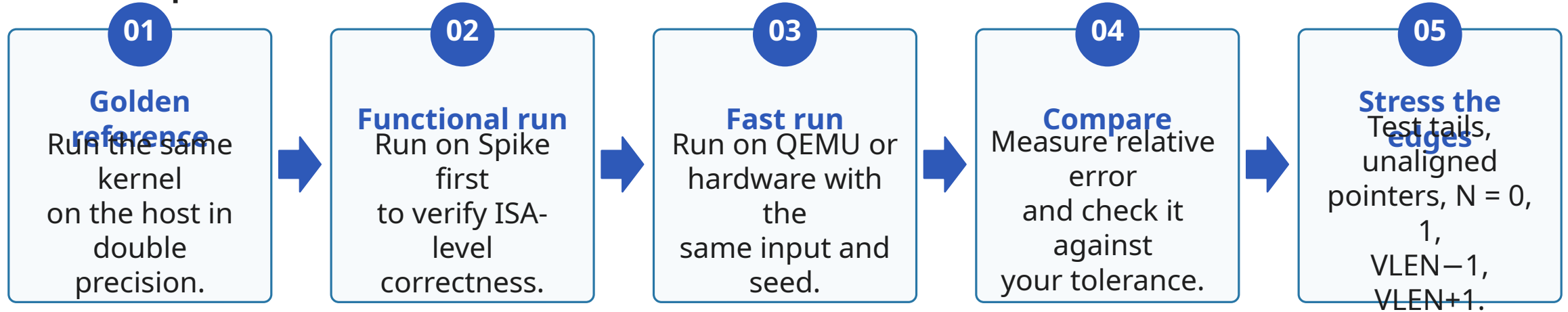
Fault Isolation by Symptom

	OBSERVED SYMPTOM	FIRST CHECK	LIKELY LAYER
START Does execution begin correctly? ↓	Illegal vector instruction UART remains silent	mstatus; readelf -A GPIO at crt0	ISA / startup clock / reset
COMPLETE Does the run finish? ↓	Accelerator never completes Trap handler repeats	STATUS; CYCLES mcause; mepc; mtval	driver / fence exceptions
CORRECT Does output match the reference? ↓	Globals are invalid Host and target differ DMA corrupts buffers	map; AT> layout compare with tolerance descriptor; buffers	link / startup numerics coherency
REPEAT / SCALE Is the result repeatable and fast?	Vector code has no speed-up Works once, then fails	cycles per element repeat; reset state	memory / tails synchronization

Validation: Is the Result Numerically Right?

Validation Prove the RVV result matches a trusted reference within a stated tolerance.

The five-step check



What counts as "correct"?

Small floating-point differences are normal. Use a declared tolerance

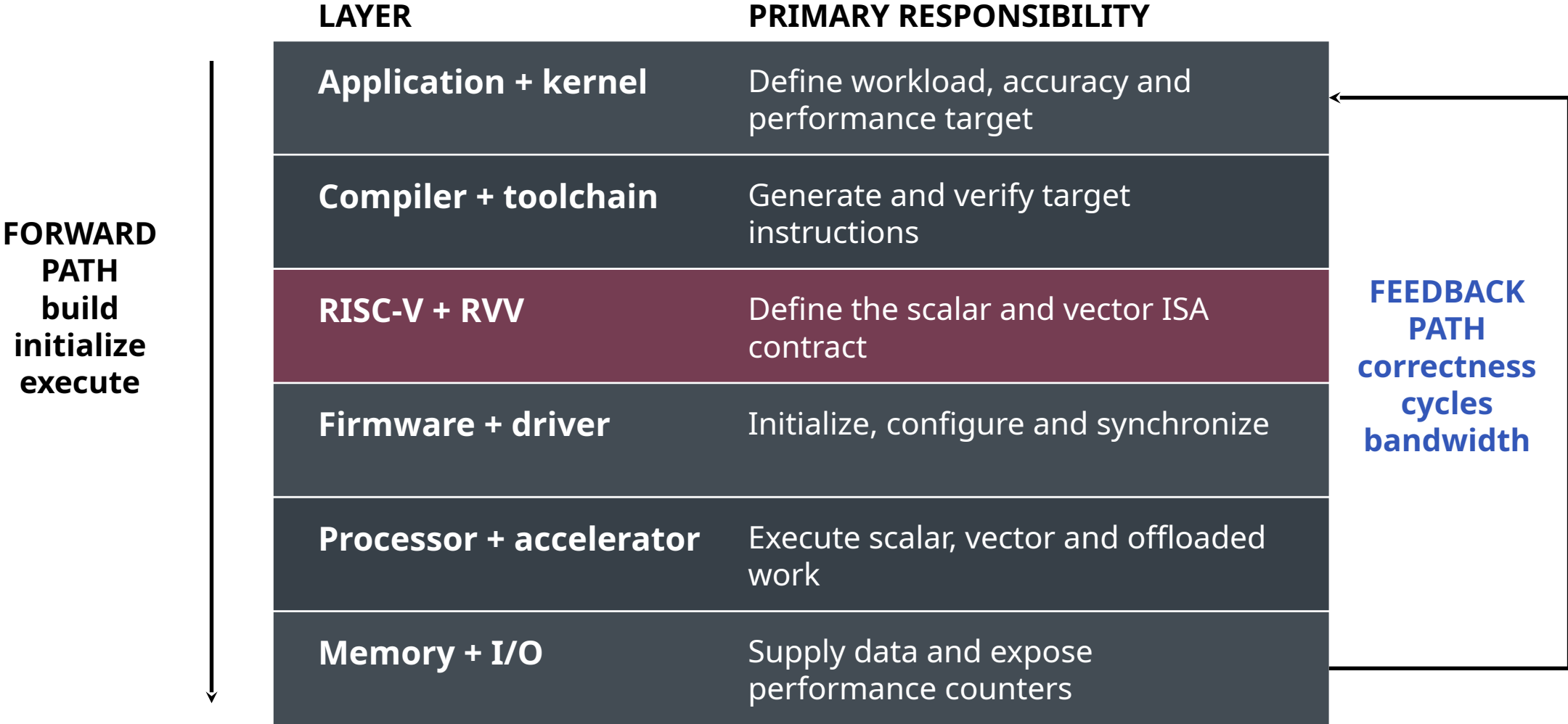
Rule before performance

Performance must pass first. Record compiler, -march, VLEN, simulator/hardware version, and tolerance.

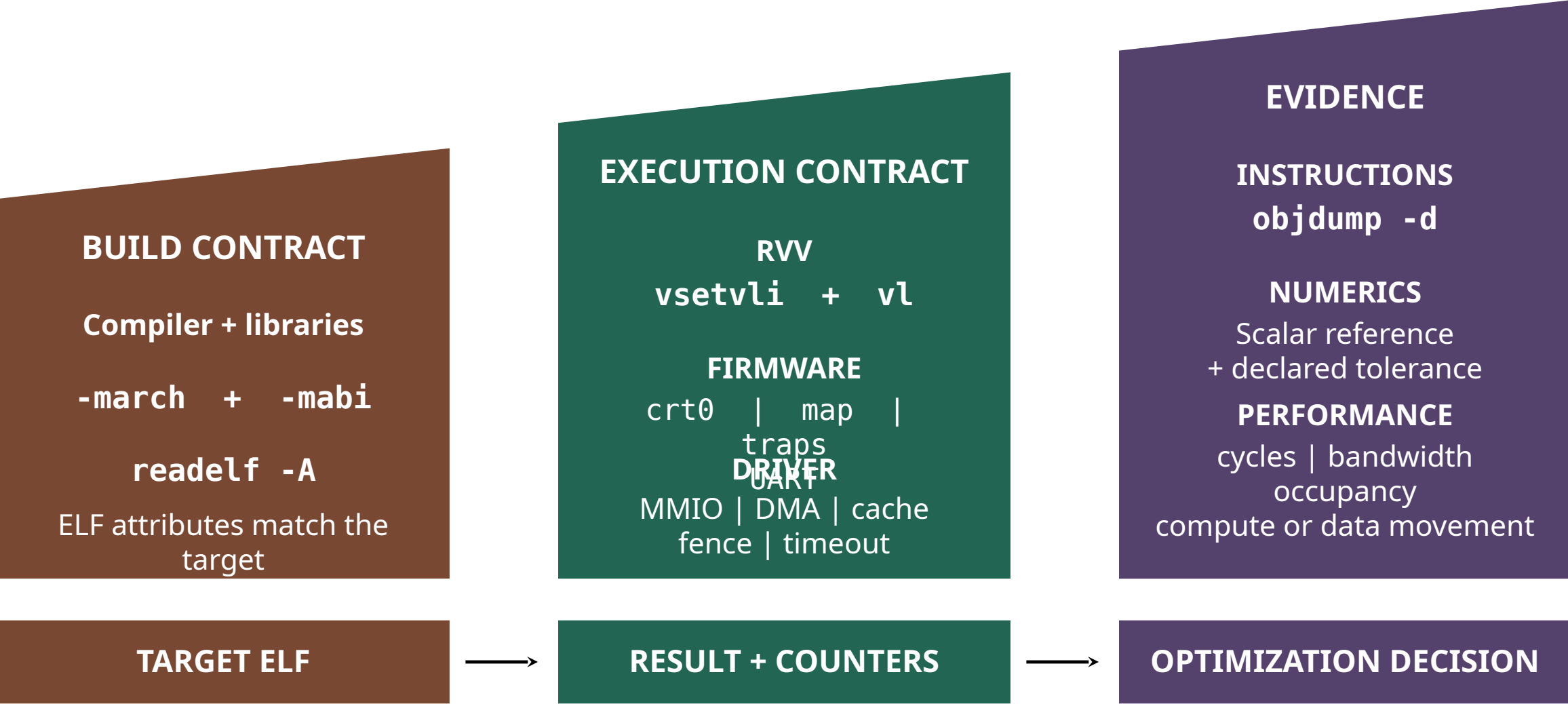
REFERENCE → TARGET → COMPARE → EDGE CASES → REPORT

Same input. Same seed. Stated tolerance.

RISC-V Vector and Accelerator Full Stack



Software-Hardware Co-Design Principles



Key Takeaways



- 01 TOOLCHAIN** `readelf -A`
Build and inspect an ELF
- 02 RVV** `objdump -d`
Vectorise one kernel
- 03 FIRMWARE** `boot / trap log`
Boot, UART and traps
- 04 ACCELERATOR** `STATUS + CYCLES`
MMIO, DMA and completion
- 05 PERFORMANCE** `metrics table`
Measure cycles and break-even



Questions

Thank you